Hummingbird API – Midterm Bug Report

Ticket #1 — "Server started on the wrong port" *(Easy)*

Reproduction:

```
humming-bird-midterm $ curl $ALB/health
{"status":"ok","service":"hummingbird","timestamp":1772680111474}
```

Note that the health endpoint works even if the code doesn't have a fallback.

Explanation:
There is no fallback if APP_PORT isn't set, port is undefined.

Where is it actually being set?

terraform/ecs.tf:65:
{ name = "APP_PORT", value = tostring(var.app_port) }

var.app_port is defined in terraform/variables.tf:25-29:

```
variable "app_port" {
  description = "Container/listener port the API listens on."
  type        = number
  default     = 9000
}
```

Bug Report

File: humming-bird-midterm/server.js, line 35
const port = process.env.APP_PORT;

The bug: There is no fallback value for the port. If the APP_PORT environment variable is not set, port becomes undefined.

Why it's broken: When app.listen(undefined) is called, Node.js will assign a random ephemeral port instead of the expected one. This directly conflicts with the Dockerfile (lines 17 and 23-24), which hardcodes port 9000 in both the EXPOSE directive and the HEALTHCHECK. If the container or app ever starts without APP_PORT explicitly passed in, the health check will fail (it hits localhost:9000) and the container will be marked unhealthy even though the server is technically running, just on a random port.

Fix: Add a default that matches the Dockerfile's expectation:

```
const port = process.env.APP_PORT || 9000;
```

Diff of the change:

```
--- a/humming-bird-midterm/server.js
+++ b/humming-bird-midterm/server.js
@@ -32,7 +32,7 @@ app.get('/health', (req, res) => {

 app.use('/v1/media', mediaRoutes);

-const port = process.env.APP_PORT;
+const port = process.env.APP_PORT || 9000;

 app.listen(port, () => {
   logger.info(`Example app listening on port ${port}`);
```

Server.js now uses port 9000

Ticket #2 — "Width is missing from metadata" *(Easy)*

Reproduction

```
● Bash(curl -s "http://hummingbird-production-alb-200222969.us-west-2.elb.amazonaws.com/v1/media/a395c49a-b5a5-41d0-9a8d-cc16fb89b083" | python3 -m json.tool)
  └ {
        "mediaId": "a395c49a-b5a5-41d0-9a8d-cc16fb89b083",
        "size": 149372,
        "name": "sample.jpg",
        "mimetype": "image/jpeg",
        "status": "PENDING"
    }
```

Bash(aws logs tail /ecs/hummingbird/production/api --since 5m --format short 2>&1 | tail -30)

```
{"level":"info","message":{"duration":1157,"method":"POST","statusCode":202,"url":"/v1/media/upload?width=800"},"service":"hummingbird","timestamp":"2026-03-05T03:41:11.900Z"}
2026-03-05T03:41:11 {"level":"warn","message":{"type":"media.v1.resize-media"},"service":"hummingbird","timestamp":"2026-03-05T03:41:11.962Z"}
```

Bug Report

File: clients/dynamodb.js, lines 78-84

```
return {
  mediaId,
  size: Number(Item.size.N),
  name: Item.name.S,
  mimetype: Item.mimetype.S,
  status: Item.status.S,
};
```

The bug: The getMedia function reads a DynamoDB item that contains width (written by createMedia on line 34), but the return object never includes it. The field exists in the database it's simply never mapped back into the response.

Why it's broken: When getController calls getMedia and returns the result to the client, width is silently dropped. The user uploads with ?width=800, the value is persisted correctly, but GET /v1/media/:id never shows it because getMedia doesn't extract Item.width.N from the DynamoDB response.

Fix: Add width to the return object:

```
return {
  mediaId,
  size: Number(Item.size.N),
  name: Item.name.S,
  mimetype: Item.mimetype.S,
  status: Item.status.S,
  width: Number(Item.width.N),
};
```

Diff of the change:

```
--- a/humming-bird-midterm/clients/dynamodb.js
+++ b/humming-bird-midterm/clients/dynamodb.js
@@ -78,6 +78,7 @@
      size: Number(Item.size.N),
      name: Item.name.S,
      mimetype: Item.mimetype.S,
      status: Item.status.S,
+     width: Number(Item.width.N),
    };
```

Verification:

```
humming-bird-midterm $ curl AWS CloudShell B/v1/media/upload?width=800" -F "file=@sample.jpg"
{"mediaId":"745aaa8b-1881-4a68-82c6-8c7246704ed4"}humming-bird-midterm $
humming-bird-midterm $ curl $ALB/v1/media/745aaa8b-1881-4a68-82c6-8c7246704ed4
{"mediaId":"745aaa8b-1881-4a68-82c6-8c7246704ed4","size":149372,"name":"sample.jpg","mimetype":"image/jpeg","status":"PENDING","width":800}
```

Note that now “width” is included in the response.

Ticket #3 — "Your redirect URL is broken" *(Intermediate)*

Before:

Notice here Location header has no “http:” prefixed

Other observations: Says in progress but status is still pending

Bug Report

File: controllers/media.js, line 111

res.set('Location', `${req.hostname}/v1/media/${mediaId}/status`);

The bug: The Location header is missing the http:// protocol prefix. req.hostname returns just the bare hostname (e.g., hummingbird-production-alb-200222969.us-west-2.elb.amazonaws.com), so the resulting header value is:

Location: hummingbird-production-alb-200222969.us-west-2.elb.amazonaws.com/v1/media/.../status

Why it's broken: Without http://, this is not a valid URL. HTTP clients that receive a 202 with a Location header can't follow it to check the media's status. Additionally, req.hostname strips the port if the service were

accessed on a non-standard port, the URL would also be missing that. req.get('host') should be used instead, which preserves the port.

Fix:
res.set('Location', `http://${req.get('host')}/v1/media/${mediaId}/status`);

Diff of the change:

```
--- a/humming-bird-midterm/controllers/media.js
 +++ b/humming-bird-midterm/controllers/media.js
 @@ -108,7 +108,7 @@
      const SIXTY_SECONDS = 60;
      res.set('Retry-After', SIXTY_SECONDS);
 -    res.set('Location', `${req.hostname}/v1/media/${mediaId}/status`);
 +    res.set('Location',
`http://${req.get('host')}/v1/media/${mediaId}/status`);
```

After:

```
humming-bird-midterm $ curl -i $ALB/v1/media/800bd614-b382-49ae-8e0b-c7544fcf5d8f/download
HTTP/1.1 202 Accepted
Date: Thu, 05 Mar 2026 04:30:08 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 43
Connection: keep-alive
X-Powered-By: Express
Retry-After: 60
Location: http://hummingbird-production-alb-200222969.us-west-2.elb.amazonaws.com/v1/media/800bd614-b382-49ae-8e0b-c7544fcf5d8f/status
ETag: W/"2b-DVJ8jHebx01CN9tAyv9tXcsioNc"

{"message":"Media processing in progress."}humming-bird-midterm $
```

Location now has “http” prefixed

Ticket #4 — "Download never redirects even when COMPLETE" (Intermediate)

Reproduction

Observation:

- PUT /resize returned {"status":"COMPLETE"} - the resize controller thinks it updated the status
  - But GET /status still says {"status":"PENDING"} - the status never actually changed

Bug Report

File: controllers/media.js, line 108

```
if (media.status !== MEDIA_STATUS.PROCESSING) {
```

The bug: The condition is inverted. It checks if the status is NOT PROCESSING to decide whether to return a 202. This means:

- PENDING → not PROCESSING → returns 202 (happens to be correct)
- PROCESSING → equals PROCESSING → falls through to 302 redirect (wrong — still processing)
- COMPLETE → not PROCESSING → returns 202 (wrong — should redirect)

Why it's broken: A COMPLETE media will never get a 302 redirect to the presigned S3 download URL. The download endpoint is permanently stuck

returning 202 "Media processing in progress" even though the media is ready. Meanwhile, media that's actively PROCESSING would incorrectly get the redirect.

Fix: Change the condition to check for !== COMPLETE:

```
 if (media.status !== MEDIA_STATUS.COMPLETE) {
```

Diff of the change:

```
● --- a/humming-bird-midterm/controllers/media.js
  +++ b/humming-bird-midterm/controllers/media.js
  @@ -108,7 +108,7 @@
  -   if (media.status !== MEDIA_STATUS.PROCESSING) {
  +   if (media.status !== MEDIA_STATUS.COMPLETE) {
```

Verification

```
humming-bird-midterm $ curl -X POST "$ALB/v1/media/upload?width=500" -F "file=@sample.jpg"
{"mediaId":"aa4e499e-a216-49b6-925e-2aff90b7e583"}humming-bird-midterm $
humming-bird-midterm $
humming-bird-midterm $ url -X PUT "$ALB/v1/media/aa4e499e-a216-49b6-925e-2aff90b7e583/resize?width=500"
-bash: url: command not found
humming-bird-midterm $ curl -X PUT "$ALB/v1/media/aa4e499e-a216-49b6-925e-2aff90b7e583/resize?width=500"
{"mediaId":"aa4e499e-a216-49b6-925e-2aff90b7e583","status":"COMPLETE"}humming-bird-midterm $
humming-bird-midterm $
humming-bird-midterm $ curl $ALB/v1/media/aa4e499e-a216-49b6-925e-2aff90b7e583/status
{"status":"PENDING"}humming-bird-midterm $
```

Observation: Though the code is fixed, it doesn't appear to be in the output.

Claudes Reasoning

The chain of failure:

1. PUT /resize calls setMediaStatus → writes COMPLETE to SK: 'metadata' (wrong item)
2. resizeController returns {"status":"COMPLETE"} because it trusts what it just wrote
3. GET /status calls getMedia → reads from SK: 'METADATA' (original item) → still PENDING
4. GET /download calls getMedia → gets PENDING → PENDING !== COMPLETE → returns 202

Conclusion: The Ticket #4 condition fix (!== COMPLETE) is correct and deployed, but it can never trigger a 302 because the status on the actual record never reaches COMPLETE. The Bonus ticket fix is required to fully verify Ticket #4.

CloudWatch doesn’t give any 302 redirection at this moment.

Bash(aws logs tail /ecs/hummingbird/production/api --since 10m --format short 2>&1 | grep -i "download\|302\|redirect\|COMPLETE\|presigned" | tail -15)
⎿ 2026-03-05T04:41:40 {"level":"info","message":{"mediaId":"aa4e499e-a216-49b6-925e-2aff90b7e583","newStatus":"COMPLETE","sk":"metadata"},"service":"hummingbird","timestamp":"2026-03-0
5T04:41:40.128Z"}
2026-03-05T04:43:30 {"level":"info","message":{"mediaId":"b58c5614-9348-490e-8487-d863637476a1","newStatus":"COMPLETE","sk":"metadata"},"service":"hummingbird","timestamp":"2026-03-0
… +3 lines (ctrl+o to expand)

No 302 anywhere. Here's what the logs confirm:

1. setMediaStatus writes COMPLETE to sk: "metadata" (lowercase) the wrong DynamoDB item
2. GET /download still returns statusCode: 202 because getMedia reads from SK: "METADATA" (uppercase), where the status is still PENDING

There are zero 302 responses or "Media is COMPLETE, generating presigned URL" log entries. The Ticket #4 fix is deployed and correct, but it will never produce a 302 until the Bonus ticket's SK casing mismatch is fixed.

Bonus — "Status never changes. No errors. Nothing." *(Advanced)*

Reproduction

There are 2 entries one with “METADATA” other with “metadata”

Check specifically for dynamo db and metadata in the logs:

Diff of the change:

```
--- a/humming-bird-midterm/clients/dynamodb.js
+++ b/humming-bird-midterm/clients/dynamodb.js
@@ -154,7 +154,7 @@
       PK: { S: `MEDIA#${mediaId}` },
-      SK: { S: 'metadata' },
+      SK: { S: 'METADATA' },
```

Verification: